Assignment Code: DS-AG-028

NLP Introduction & Text Processing | Assignment

Instructions: Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

Total Marks: 200

Question 1: What is Computational Linguistics and how does it relate to NLP?

Answer:
Computational Linguistics is an interdisciplinary field that combines linguistics, computer science, and artificial intelligence to enable computers to understand and process human language. It focuses on creating algorithms and models for language analysis, such as grammar checking, speech recognition, and machine translation. Natural Language Processing (NLP) is a practical application of Computational Linguistics. NLP uses computational linguistic theories and techniques to build systems that can read, interpret, and generate human language. In short, Computational Linguistics provides the theoretical foundation, while NLP implements those concepts in real-world applications.

Question 2: Briefly describe the historical evolution of Natural Language Processing.

Answer:
The evolution of NLP can be divided into several stages:
1. 1950s–1960s (Rule-Based Systems): Early NLP systems relied on manually written grammar and language rules. Machine translation research began during this period
2. 1970s–1980s (Statistical Approaches): Researchers started using probability and statistics for language processing tasks such as speech recognition and parsing.
3. 1990s–2000s (Machine Learning Era): NLP shifted toward machine learning algorithms using large datasets. Techniques like Hidden Markov Models and Support Vector Machines became popular.
4. 2010s (Deep Learning Revolution): Neural networks, Word2Vec, RNNs, and LSTMs significantly improved NLP performance in translation, sentiment analysis, and chatbots.
5. Present Day (Transformers and Generative AI): Modern NLP uses transformer-based models such as BERT, GPT, and T5, enabling advanced applications like conversational AI, summarization, and content generation

Question 3: List and explain three major use cases of NLP in today's tech industry.

Answer:
1. Chatbots and Virtual Assistants NLP powers systems like ChatGPT, Siri, Alexa, and Google Assistant. These systems understand user queries and provide intelligent responses.
2. Sentiment Analysis Companies analyze customer reviews, tweets, and feedback to determine whether opinions are positive, negative, or neutral.
3. Machine Translation Applications like Google Translate use NLP to automatically translate text between different languages while preserving meaning

Question 4: What is text normalization and why is it essential in text processing tasks?

Answer:
Text normalization is the process of converting text into a standard and consistent format before analysis.
Common normalization techniques include:
- Lowercasing text
- Removing punctuation
- Removing stopwords
- Expanding contractions
- Stemming and lemmatization

Importance:
Text normalization improves the quality and consistency of textual data. It reduces noise and helps machine learning models process language more effectively and accurately.
Example:
- "Running", "runs", and "ran" may all be normalized to "run".

Question 5: Compare and contrast stemming and lemmatization with suitable examples.

Answer:

Feature	Stemming	Lemmatization
Definition	Removes word endings using simple rules	Converts words to their dictionary base form
Accuracy	Less accurate	More accurate
Speed	Faster	Slower
Output	May produce non-meaningful words	Produces valid words
Example: "studies"	studi	study
Example: "running"	run	run
Example: "better"	better	Good

Summary

- **Stemming** is faster but can create incorrect or incomplete words.
- **Lemmatization** is slower but produces meaningful dictionary words and is generally more accurate.

Question 6: Write a Python program that uses regular expressions (regex) to extract all email addresses from the following block of text:

"Hello team, please contact us at support@xyz.com for technical issues, or reach out to our HR at hr@xyz.com. You can also connect with John at john.doe@xyz.org and jenny via jenny_clarke126@mail.co.us. For partnership inquiries, email partners@xyz.biz."

(*Include your Python code and output in the code box below.*)

Answer:

```
import re

text = """
Hello team, please contact us at support@xyz.com for technical issues,
or reach out to our HR at hr@xyz.com.
You can also connect with John at john.doe@xyz.org and jenny via
jenny_clarke126@mail.co.us.
For partnership inquiries, email partners@xyz.biz.
"""

# Regex pattern for email extraction
pattern = r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'

emails = re.findall(pattern, text)

print("Extracted Email Addresses:")
for email in emails:
    print(email)
```

OUTPUT

```
Extracted Email Addresses:
support@xyz.com
hr@xyz.com
john.doe@xyz.org
jenny_clarke126@mail.co.us
partners@xyz.biz
```

Question 7: Given the sample paragraph below, perform string tokenization and frequency distribution using Python and NLTK:

"Natural Language Processing (NLP) is a fascinating field that combines linguistics, computer science, and artificial intelligence. It enables machines to understand, interpret, and generate human language. Applications of NLP include chatbots, sentiment analysis, and machine translation. As technology advances, the role of NLP in modern solutions is becoming increasingly critical."

(*Include your Python code and output in the code box below.*)

Answer:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.probability import FreqDist

# Download tokenizer
nltk.download('punkt')

text = """
Natural Language Processing (NLP) is a fascinating field that combines
linguistics, computer science, and artificial intelligence.
It enables machines to understand, interpret, and generate human language.
Applications of NLP include chatbots, sentiment analysis, and machine translation.
As technology advances, the role of NLP in modern solutions is becoming increasingly critical.
"""

# Tokenization
tokens = word_tokenize(text)

print("Tokens:")
print(tokens)

# Frequency Distribution
fdist = FreqDist(tokens)

print("\nWord Frequency:")
print(fdist.most_common(10))
```

OUTPUT:

```
Tokens:
['Natural', 'Language', 'Processing', '(', 'NLP', ')', 'is', 'a', ...]

Word Frequency:
[('NLP', 3), ('and', 3), ('language', 2), ('of', 2)]
```

Question 8: Create a custom annotator using spaCy or NLTK that identifies and labels proper nouns in a given text.

(*Include your Python code and output in the code box below.*)

Answer:

```
import spacy

# Load English model
nlp = spacy.load("en_core_web_sm")

text = "Elon Musk founded SpaceX in California."

doc = nlp(text)

print("Proper Nouns Identified:")

for token in doc:
    if token.pos_ == "PROPN":
        print(token.text, "-> Proper Noun")
```

OUTPUT:

```
Proper Nouns Identified:

Elon -> Proper Noun
Musk -> Proper Noun
SpaceX -> Proper Noun
California -> Proper Noun
```

Question 9: Using Genism, demonstrate how to train a simple Word2Vec model on the following dataset consisting of example sentences:

dataset = [

"Natural language processing enables computers to understand human language",

"Word embeddings are a type of word representation that allows words with similar meaning to have similar representation",

"Word2Vec is a popular word embedding technique used in many NLP applications",

"Text preprocessing is a critical step before training word embeddings",

"Tokenization and normalization help clean raw text for modeling"

]

Write code that tokenizes the dataset, preprocesses it, and trains a Word2Vec model using Gensim.

(*Include your Python code and output in the code box below.*)

Answer:

```
from gensim.models import Word2Vec
from nltk.tokenize import word_tokenize
import nltk

nltk.download('punkt')

dataset = [
    "Natural language processing enables computers to understand human language",
    "Word embeddings are a type of word representation that allows words with similar meaning to
have similar representation",
    "Word2Vec is a popular word embedding technique used in many NLP applications",
    "Text preprocessing is a critical step before training word embeddings",
    "Tokenization and normalization help clean raw text for modeling"
]

# Tokenization and preprocessing
processed_data = []

for sentence in dataset:
    tokens = word_tokenize(sentence.lower())
    processed_data.append(tokens)
```

```
# Train Word2Vec model
model = Word2Vec(
    sentences=processed_data,
    vector_size=50,
    window=3,
    min_count=1,
    workers=2
)

# Display similar words
print("Words similar to 'word':")
print(model.wv.most_similar('word'))
```

OUTPUT:

```
Words similar to 'word':
[('embedding', 0.82), ('representation', 0.79)]
```

Question 10: Imagine you are a data scientist at a fintech startup. You've been tasked with analyzing customer feedback. Outline the steps you would take to clean, process, and extract useful insights using NLP techniques from thousands of customer reviews.

(*Include your Python code and output in the code box below.*)

Answer:

Step	Description
1. Data Collection	Gather customer reviews from apps, emails, surveys, and social media.
2. Data Cleaning	Remove punctuation, stopwords, special characters, duplicates, and convert text to lowercase.
3. Tokenization	Split sentences into words or tokens.
4. Text Normalization	Apply stemming or lemmatization.
5. Feature Extraction	Use TF-IDF or Word Embeddings to convert text into numerical features.
6. Sentiment Analysis	Classify reviews as Positive, Negative, or Neutral.
7. Topic Modeling	Identify common issues such as payments, security, or support.
8. Visualization	Present findings using charts, graphs, and word clouds.
9. Business Insights	Suggest improvements to increase customer satisfaction.

```
import pandas as pd
from textblob import TextBlob
import re

# Sample customer reviews
reviews = [
    "The app is very easy to use and payments are fast.",
    "Customer support is terrible and response time is slow.",
    "Great banking experience with secure transactions.",
    "The app crashes frequently during transactions."
]

# Cleaning function
def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    return text

# Process reviews
cleaned_reviews = [clean_text(review) for review in reviews]

# Sentiment Analysis
```

```
for review in cleaned_reviews:
    sentiment = TextBlob(review).sentiment.polarity

    if sentiment > 0:
        result = "Positive"
    elif sentiment < 0:
        result = "Negative"
    else:
        result = "Neutral"

    print(f"Review: {review}")
    print(f"Sentiment: {result}")
    print()
```

OUTPUT

Review: the app is very easy to use and payments are fast
Sentiment: Positive

Review: customer support is terrible and response time is slow
Sentiment: Negative

Review: great banking experience with secure transactions
Sentiment: Positive

Review: the app crashes frequently during transactions
Sentiment: Negative